

Architecture — Wash Manager

Vue d'ensemble (4 couches)

```
flowchart LR
    subgraph Captation
        C1[Caméra Entrée + LPR]
        C2[Caméras Zones B/C/D]
        C3[Caméra Sortie]
        NFC[Lecteurs NFC employés]
    end
    end
    subgraph Traitement["Traitement IA – serveur edge (ai/)"]
        DET[YOLO détection]
        TRK[Tracking track_id]
        LPR[OCR plaques MA]
        ZON[Zones & lignes]
        ORC[Orchestrateur → events]
    end
    end
    subgraph Backend["Backend (backend/)"]
        ING[Ingestion events]
        CLF[Classification forfait]
        ANO[Détection anomalies]
        RAP[Rapport journalier]
        DB[(PostgreSQL)]
        API[API REST + WebSocket]
    end
    end
    subgraph Visualisation
        DASH[Dashboard web / PWA]
        WA[Alertes WhatsApp]
    end
    end

    C1 & C2 & C3 --> DET --> TRK --> ZON --> ORC
    C1 --> LPR --> ORC
    NFC --> ORC
    ORC --> |POST /events| ING --> DB
    ING --> CLF --> ANO --> DB
    RAP --> DB
    ANO --> WA
    DB --> API --> DASH
```

Séquence — passage d'un véhicule

```
sequenceDiagram
    participant V as Véhicule
    participant AI as Pipeline IA (ai/)
    participant BE as Backend (backend/)
    participant CA as Caisse intégrée (dashboard)

    V->>AI: franchit ligne entrée
    AI->>BE: event ENTREE (track_id)
    BE->>BE: crée transaction "en_cours"
    AI->>BE: event PLAQUE (LPR)
    BE->>BE: associe/crée véhicule
    V->>AI: présence zone B/C/D
    AI->>BE: events ZONE_ENTER / ZONE_EXIT
    BE->>BE: chrono par zone
    V->>AI: franchit ligne sortie
    AI->>BE: event SORTIE
    BE->>BE: classification forfait effectué
    CA-->>BE: ticket créé à l'encaissement (forfait payé)
    BE->>BE: rapprochement ticket + détection anomalies
    BE-->>BE: si CRITIQUE/HAUTE → alerte whatsapp
```

Répartition des responsabilités

- ai/ ne fait que percevoir : détecter, suivre, lire, situer. Aucune règle

métier. Il émet des **faits** (events).

- backend/ **décide** : construit les transactions, classe le forfait, applique les règles d'anomalies, produit les rapports, expose l'API.
- frontend/ **affiche** : consomme l'API, ne contient pas de logique métier.

Ce découplage permet de développer/tester chaque couche indépendamment et de déployer le pipeline IA sur une machine edge séparée du serveur applicatif.

Interface pivot : les événements

Le contrat d'événement (`backend/app/schemas/event.py` ↔ `ai/pipeline/events.py`) est le point d'intégration central. Toute évolution du pipeline (nouvelle zone, nouveau capteur) passe par un nouveau type/champ d'événement, sans toucher au reste.